

Computational and AI Methods in Climate Economics

Lecture 2 (Part 1) — Surrogate models and Gaussian processes for economics and finance

Simon Scheidegger

HEC Lausanne · Grantham Research Institute, LSE

July 15, 2026 · CIRM, Marseille, France

Based on: Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*; Friedl et al. (2023); Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*)

<https://cemracs2026.math.cnrs.fr/en/>

Roadmap of This Lecture

Main part — surrogate models in economics and finance:

- ▶ Why surrogates? From lookup tables to deep neural networks
- ▶ Pseudo-states: model parameters as extra inputs
- ▶ Building, validating, and stress-testing a surrogate
- ▶ Applications: structural estimation, uncertainty quantification, optimal policy, option pricing
- ▶ A pitfall to know: **lazy learning** — and how to fix it

Appendix — the Gaussian-process toolbox:

- ▶ GP regression and deep kernels
- ▶ Bayesian active learning

Hands-on: `01_Surrogate_Primer.ipynb` (main part); notebooks `02_GP_and_BAL.ipynb` and `03_Deep_Kernel_Learning.ipynb` accompany the appendix.

Learning objectives

By the end of this lecture you should be able to:

- ▶ Explain what a surrogate model is and why economics and finance are ideal terrain for them (we own the data-generating model)
- ▶ Design and train deep surrogates that approximate expensive economic models by treating parameters as pseudo-states
- ▶ Compare approximation methods (polynomials, sparse grids, GPs, DNNs) and recognize when each is appropriate
- ▶ Sketch the main use cases: structural estimation, uncertainty quantification, optimal policy design, and option-pricing calibration
- ▶ Diagnose **lazy learning** in residual-trained surrogates and apply the anchor/distillation and residual-correction fixes

The Gaussian-process machinery behind several of these applications (GP regression, active learning) is developed in the **appendix** and in notebooks `02_GP_and_BAL.ipynb` and `03_Deep_Kernel_Learning.ipynb`.

Motivation: Why Surrogates?

The computational bottleneck

Economic & financial models are expensive to evaluate:

- ▶ Global solutions of dynamic stochastic models (DEQNs)
- ▶ PDE-based models (HJB, Black–Scholes for exotics)
- ▶ Monte Carlo simulations for risk measures

Key insight

We know the true model — we can generate *unlimited* training data by solving the model on a design of experiments.

Goal: Build a **surrogate** $\phi(\cdot | \theta_{NN})$ that approximates $f(\cdot)$ but evaluates *orders of magnitude faster*.

Look-up Tables $\rightarrow$ Neural Networks

Traditional: pre-compute & interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

Example: standard normal CDF table.

21st-century look-up table:

A neural network memorizes $\mathbf{x} \mapsto y$:

$$\phi(\mathbf{x}_t | \theta_{NN}) \approx y_t = f(\mathbf{x}_t)$$



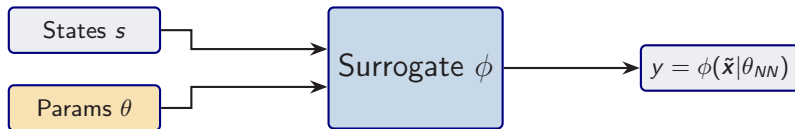
Continuous, differentiable, works in any dimension.

Pseudo-States: The Key Idea

Core insight (Chen, Didisheim & Scheidegger 2026)

Treat model parameters θ as **extra state variables**:

$$\tilde{\mathbf{x}} = \underbrace{(s_1, \dots, s_n)}_{\text{states}}, \underbrace{(\theta_1, \dots, \theta_p)}_{\text{parameters}} \in \mathbb{R}^d, \quad d = n + p.$$



Solve the model **once** over the full augmented space $\Rightarrow$ instant evaluation for *any* parameter configuration.

Building the Surrogate

1. **Design:** Draw $\tilde{\mathbf{x}}_i$ uniformly from $[\underline{x}, \bar{x}]^d$, $i = 1, \dots, m$.
2. **Label:** Compute $y_i = f(\tilde{\mathbf{x}}_i)$ using the expensive model.
3. **Train:** Minimize the empirical risk:

$$\hat{\theta}_{NN} = \arg \min_{\theta_{NN}} \frac{1}{m} \sum_{i=1}^m |\phi(\tilde{\mathbf{x}}_i | \theta_{NN}) - y_i|^2.$$

4. **Validate:** On a held-out test set, compute max / mean absolute error.

Remark

Since we can generate *unlimited* data from the true model, classical overfitting from data scarcity is not the binding constraint. The binding errors come from **approximation** (network capacity vs. true regularity of f) and from **label noise** when f is itself an approximate inner solver (Euler residuals, Monte Carlo simulation).

Why Surrogates $\neq$ Standard ML

Standard supervised learning:

- ▶ Data is *scarce* and noisy
- ▶ Overfitting is a major concern
- ▶ Regularization, early stopping, cross-validation
- ▶ Small learning rates, careful tuning

Surrogate modeling:

- ▶ Data is **unlimited** (we own the model)
- ▶ Labels are **exact** (or noise-free)
- ▶ Can use large epochs, aggressive LR
- ▶ Focus on *approximation error*, not generalization

Once trained

- ▶ **Accurate enough for the downstream task** (relative errors typically 10^{-3} – 10^{-5} on benchmarks; verify per application)
- ▶ **Orders of magnitude faster** than the original model
- ▶ **Differentiable**: gradients via automatic differentiation (backprop)

Speed Comparison: Option Pricing

From Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*:

Task	FFT	Surr. (CPU)	Surr. (GPU)
Single price	23 ms	0.03 ms	—
1-day calib. (6×10^5)	3.3 h	18 s	0.4 s
1-year calib. (1.5×10^8)	≈ 100 d	1.2 h	98 s

Takeaway

Deep surrogates enable **real-time calibration** of option pricing models — tasks that previously took days or were infeasible.

Comparison of Approximation Methods

Criterion	Polynomials/Splines	Sparse Grids	GPs	DNNs
High raw dim. ($d > 10$, no AS)	✗	~	✗	✓
Local features / kinks	✗	✓	~	✓
Irregular domains	✗	✗	✓	✓
Large training set ($N > 10^4$)	✓	~	✗	✓
Built-in uncertainty	✗	✗	✓	✗

- DNNs excel in high dimensions with large data.
- GPs provide **built-in uncertainty quantification** — crucial for active learning and Bayesian inference.
- **Combining both** (GP for moderate d with UQ, DNN for high d) is a powerful strategy.

Application I: Structural Estimation

Goal: Estimate structural parameters θ from data via SMM:

$$\hat{\theta}_{SMM} = \arg \min_{\theta} [m(\theta) - \hat{m}^{\text{data}}]^{\top} W [m(\theta) - \hat{m}^{\text{data}}],$$

where $m(\theta)$ are model-implied moments and computing them naively requires solving and simulating the model for each candidate θ .

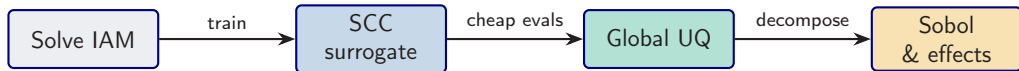
Surrogate-based estimation

1. Train a surrogate policy $\phi(x, \theta)$ over **states and parameters** (pseudo-states).
2. Reuse the trained surrogate to simulate moments $m(\theta)$ without re-solving the model.
3. Evaluate the SMM criterion over many θ values via grid search or standard optimization.

Result: Repeated SMM evaluations in **seconds** instead of weeks.
(Chen, Didisheim & Scheidegger 2026)

Application II: Uncertainty Quantification

DeepUQ (Friedl, Kübler, Scheidegger & Usui 2023):



- ▶ Solve a high-dimensional IAM, train a surrogate for the **social cost of carbon** $\text{SCC}(\theta)$
- ▶ Cheap surrogate evaluations enable **global sensitivity analysis**: Sobol indices, univariate effects
- ▶ Identifies which climate/economic parameters drive SCC uncertainty

Application III: Optimal Policy

Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*):

- ▶ High-dimensional integrated assessment model (IAM)
- ▶ GP surrogate + Bayesian Active Learning (BAL) for the value function
- ▶ GP provides *uncertainty quantification* at each point
- ▶ BAL selects training points where uncertainty is highest
- ▶ Result: optimal carbon tax policy

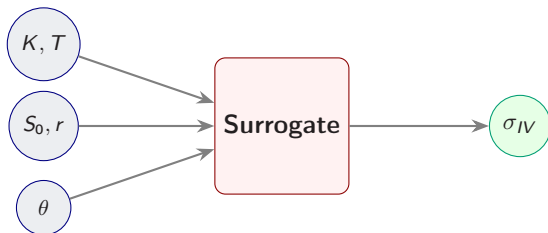
Preview

The appendix introduces Gaussian Processes and Bayesian Active Learning — the tools behind this application (Lecture 6).

Application IV: The Implied Volatility Surface

Goal: Learn the mapping from option characteristics to implied volatility.

$$\sigma_{IV} = \phi(K, T, S_0, r, \theta_{\text{model}})$$



Benefits.

- ▶ **Calibration:** fit θ by gradient descent on $\sum_i [\phi - \sigma_{IV}^{\text{mkt}}]^2$; $\partial\phi/\partial\theta$ from autograd.
- ▶ **Arbitrage-free:** differentiable penalties for calendar ($\partial_T C \geq 0$) and butterfly ($\partial_K^2 C \geq 0$) violations.
- ▶ **Speed:** real-time pricing for stochastic-vol models without closed form.

Pseudo-States: Policy Parameterization

Split the parameter vector

$\theta = (\theta_{\text{model}}, \theta_{\text{pol}})$: **model** = environment (preferences, technology); **policy** = decision rule (tax path, Taylor coefficients). Both go in as pseudo-state; optimise by autograd through ϕ :

$$\theta_{\text{pol}}^*(\theta_{\text{model}}) = \arg \max_{\theta_{\text{pol}}} \phi(s, \theta_{\text{model}}, \theta_{\text{pol}})$$

Carbon tax

- ▶ Policy: tax trajectory θ_{pol}
- ▶ Model: climate sensitivity θ_{model}
- ▶ Optimal tax for each θ_{model}

Monetary policy

- ▶ Policy: Taylor rule coefficients
- ▶ Model: preference / technology shocks
- ▶ Optimal rule via gradient descent

Key: evaluate welfare for *any* model + policy configuration instantly $\Rightarrow$ no re-solving.

Practical Tips for Surrogate Training

Architecture

- ▶ Swish activation ($x \cdot \sigma(x)$)
- ▶ 3–5 hidden layers, 128–512 neurons
- ▶ Linear output (no activation)

Training

- ▶ $m \geq 10^5$ samples (cheap to generate)
- ▶ Adam, LR 10^{-3} , reduce on plateau
- ▶ Hundreds of epochs (no overfit risk)

Validation

- ▶ Held-out test set
- ▶ Max / mean absolute error, R^2
- ▶ Scatter: prediction vs. truth

Loss functions

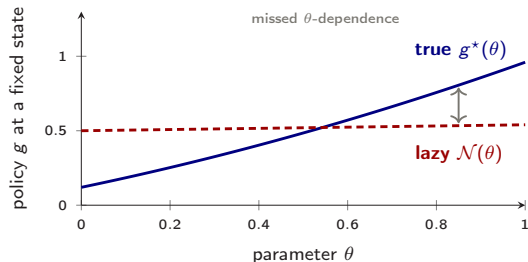
- ▶ MSE: $\frac{1}{m} \sum_i (\phi_i - y_i)^2$
- ▶ Relative L^1 : $\frac{1}{m} \sum_i |(\phi_i - y_i)/y_i|$

Pitfall: Lazy Learning — Low Residual, Wrong Policy

- **Symptom.** Lazy training is a **typical pitfall of unsupervised** (residual-only) learning (Chizat, Oyallon & Bach, 2019).
- It drives the training residual down while the policy stays inaccurate across the state–parameter domain.
- Fix a state, and vary a *parameter* θ (a taste, a tax, a shock size).
- The true policy moves with θ ; the lazy surrogate stays *flat*, so it gets the level right on average but the **sensitivity** wrong.

Too flat in the parameter

Low loss, weak parameter dependence: the surrogate is convincing only on average.



At a fixed state, the true policy varies with θ (blue); the lazy surrogate is almost flat (red dashed), so it misses how the answer should respond to the parameter.

The Fix: Anchors and Distillation

- **Fix.** Augment the self-supervised residual loss with a small **semi-supervised distillation** term (Hinton et al., 2015) on a few trusted anchor states:

$$\ell^{\text{dist}} = \ell + \lambda \sum_{\mathbf{x}_a \in \mathcal{A}} \|\mathcal{N}(\mathbf{x}_a) - p^*(\mathbf{x}_a)\|^2.$$

- $\mathcal{A}$: a handful of anchor states.
- p^* : reference policies from an accurate solver (perturbation, a low-dimensional solve, or a converged reference run).

Remedy

In the deep-UQ application of Friedl et al. (2023) — revisited in Lecture 3 — only **10–20 anchors** stabilize training, with leave-one-out agreement **within 1%**.

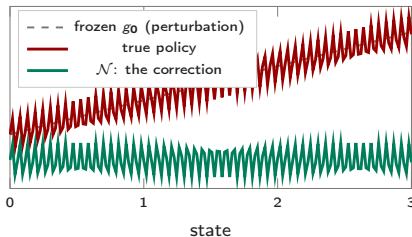
Residual Learning: Freeze and Correct

Idea. Don't ask the network for the *whole* solution. Ask for the **correction** to a cheap baseline:

$$g(\mathbf{x}) = \underbrace{g_0(\mathbf{x})}_{\text{frozen guess}} + \mathcal{N}(\mathbf{x}).$$

- ▶ g_0 : a cheap guess, steady state, perturbation, or certainty equivalent.
- ▶ The net learns only the *small, nonlinear* residual, so **spectral bias bites far less** (He et al., 2016).
- ▶ Sane from iteration zero, natural for PDE / HJB, and it scales: disaster planning in production networks (Carvalho et al., 2025).

→ **The network is a proofreader, not an author.**



The net only fits the small wiggle: frozen guess plus learned correction.

Summary: Deep Surrogate Models

1. **Surrogates** = cheap, accurate replicas of expensive models.
2. **Pseudo-states** (θ as extra inputs) unlock:
 - ▶ Structural estimation (SMM in seconds)
 - ▶ Uncertainty quantification (Sobol indices, univariate effects)
 - ▶ Optimal policy design
3. **Key advantage:** unlimited training data $\Rightarrow$ classical overfitting is not the binding constraint.
4. **Speed:** orders of magnitude faster than original model.
5. **Watch out:** residual-only training can go **lazy** — flat in the parameters despite a low loss; anchor it with a few trusted reference solutions.

Hands-on

Notebook: 01_Surrogate_Primer.ipynb — Black–Scholes surrogate with implied volatility inversion.

Questions?

Simon Scheidegger

HEC Lausanne · Grantham Research
Institute, LSE

<https://sischei.github.io>

Materials: https://github.com/sischei/cemracs2026_DEQN

Next up:

Structural Estimation with SMM

The **estimate** operator, pointed at today's surrogate.

The following appendix collects the Gaussian-process toolbox used across today's applications.

Appendix

The Gaussian-Process Toolbox

A: GP regression & deep kernels · B: Bayesian active learning

Why Gaussian Processes?

- ▶ **Nonparametric:** No fixed architecture; the model grows with data.
- ▶ **Built-in uncertainty quantification:** Posterior mean *and* variance at every point.
- ▶ **Principled kernel selection:** Encode prior beliefs about smoothness, periodicity, etc.
- ▶ **Hyperparameter learning:** Marginal likelihood provides a principled objective.

Complementary to DNNs

- ▶ GPs: best for moderate dimensions ($d \leq 10$) with built-in UQ.
- ▶ DNNs: best for high dimensions ($d \gg 10$) with large data.
- ▶ Combining both is a powerful strategy (cf. the comparison table in the main part).

Foundations: Rasmussen & Williams (2006); MacKay (1992); Krause, Singh & Guestrin (2008). In DP: Engel, Mannor & Meir (2005); Deisenroth, Rasmussen & Peters (2009). Sparse / scalable: Titsias (2009); Hensman, Fusi & Lawrence (2013).

Recall: Multivariate Gaussians

A random vector $\mathbf{x} \in \mathbb{R}^d$ is
multivariate Gaussian if

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

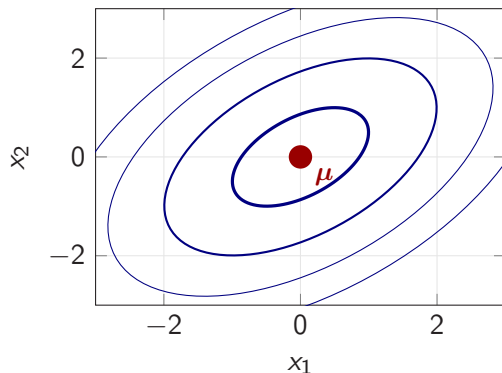
with density

$$p(\mathbf{x}) \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

Key fact

A multivariate Gaussian is **fully characterized** by its mean vector $\boldsymbol{\mu}$ and covariance matrix Σ .

2D Gaussian contours ($\rho = 0.5$)



Joint and Conditional Distributions

Joint distribution:

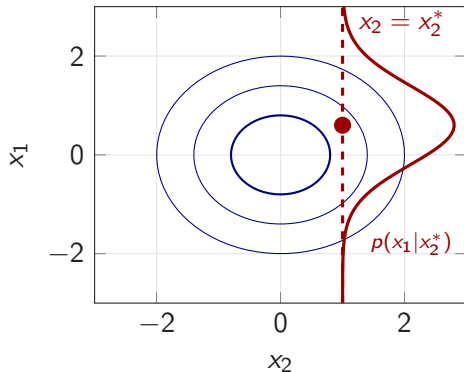
$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}\right)$$

Conditional distribution:

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

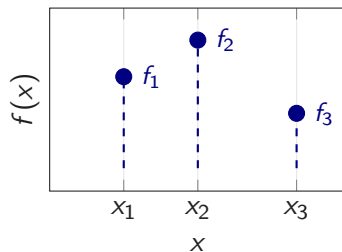
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

This is the mathematical engine behind
GP regression.



“Slicing” the joint at $x_2 = x_2^*$ yields a 1D Gaussian.

From Observations to Interpolation



Assume function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points (f_1, f_2) should be more correlated than distant ones (f_1, f_3).

Example (SE kernel, $\ell = 2$): $K = \begin{pmatrix} 1.00 & 0.76 & 0.22 \\ 0.76 & 1.00 & 0.61 \\ 0.22 & 0.61 & 1.00 \end{pmatrix}$

Covariance via a **kernel function**: $K_{ij} = k(x_i, x_j)$

$\Rightarrow$ The kernel encodes our prior belief about function smoothness.

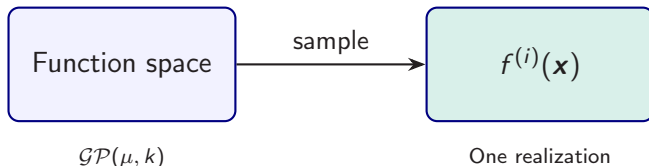
GP: A Distribution Over Functions

Definition

A **Gaussian Process** is a collection of random variables, any finite number of which have a joint Gaussian distribution:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

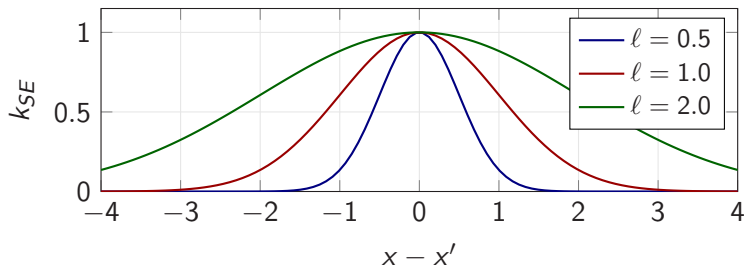
- ▶ **Mean function:** $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ (often set to 0).
- ▶ **Covariance function (kernel):** $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{x}') - \mu(\mathbf{x}'))]$.



The Squared Exponential (RBF) Kernel

$$k_{SE}(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

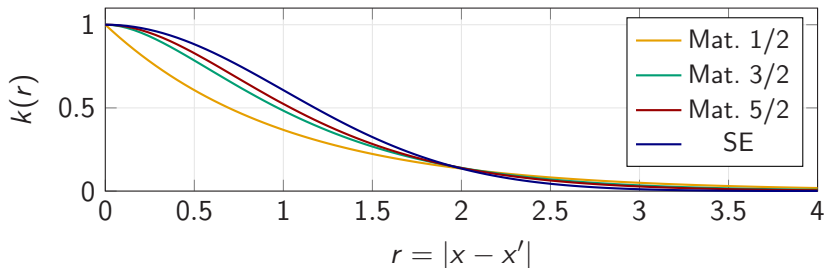
- ▶ ℓ = **length scale** (horizontal correlation range)
- ▶ σ_f^2 = **signal variance** (vertical scale)



Larger $\ell \Rightarrow$ smoother functions; smaller $\ell \Rightarrow$ more wiggly.

The Matérn Kernel Family

$$k_{\text{Mat}}(r) = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} r}{\ell} \right), \quad r = |x - x'|$$

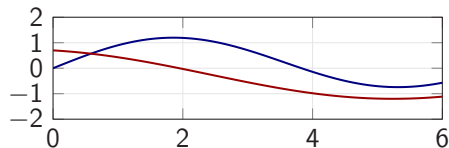


- ▶ ν controls **mean-square smoothness**: $\nu=3/2 \Rightarrow$ once MS-differentiable; $\nu=5/2 \Rightarrow$ twice; SE $\Rightarrow \infty$ -smooth.
- ▶ **Use case**: Matérn when the true function is *not* infinitely smooth (e.g., has kinks).

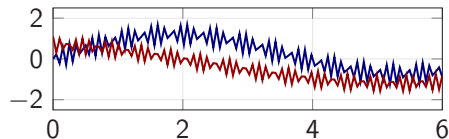
Combining Kernels

Kernels can be **added** and **multiplied** to compose complex priors from simple blocks.

SE kernel (smooth trend)



SE + Periodic (trend + seasonality)



Kernel algebra

- ▶ $k_1 + k_2$ (OR):
high if *either* is high
 $\Rightarrow$ trend + seasonality
- ▶ $k_1 \times k_2$ (AND):
high only if *both* high
 $\Rightarrow$ locally periodic

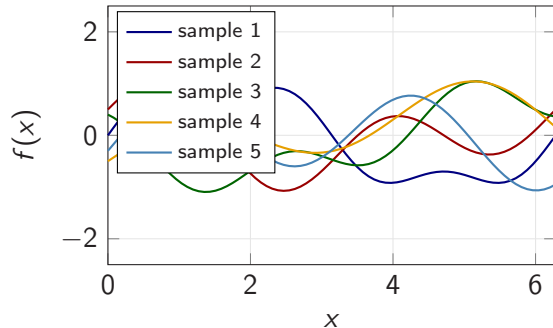
Compose **complex priors** from simple building blocks.

Sampling from the GP Prior

Algorithm

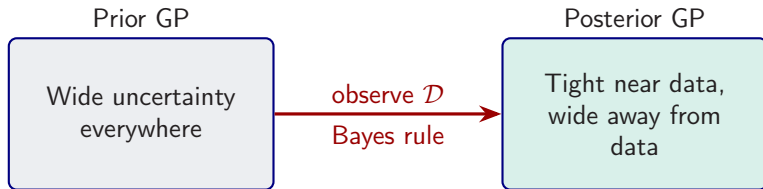
1. Grid $x_{1:N}$ over domain
2. $K_{ij} = k(x_i, x_j)$
3. Cholesky: $K = LL^T$
4. $\mathbf{f}^{(s)} = L \cdot \mathbf{u}$, $\mathbf{u} \sim \mathcal{N}(0, I)$

Five functions drawn from a GP prior with SE kernel ($\ell = 1$, $\sigma_f = 1$).



From Prior to Posterior: Bayes Rule

- ▶ **Training set:** $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$.
- ▶ **Test points:** $\mathbf{X}_*$ where we want predictions.
- ▶ Joint distribution of training outputs $\mathbf{f}$ and test outputs $\mathbf{f}_*$ is Gaussian.



The posterior is *still a GP* — fully characterized by updated mean and covariance.

Noiseless GP Regression

Joint distribution:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix}\right)$$

where $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, $(K_*)_{ij} = k(\mathbf{x}_i, \mathbf{x}_*^{(j)})$, $(K_{**})_{ij} = k(\mathbf{x}_*^{(i)}, \mathbf{x}_*^{(j)})$.

Posterior (conditioning on $\mathbf{f}$):

$$\boldsymbol{\mu}_{*|\mathcal{D}} = K_*^\top K^{-1}(\mathbf{f} - \boldsymbol{\mu}) + \boldsymbol{\mu}_* \quad (1)$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_* \quad (2)$$

Key insight

- ▶ Near observed data: $K_*^\top K^{-1} K_* \approx K_{**} \Rightarrow$ **small variance**.
- ▶ Far from data: $K_*^\top K^{-1} K_* \approx 0 \Rightarrow$ **large variance** (reverts to prior).

Prediction at a Single Test Point

Noiseless setting ($\sigma_y^2 \rightarrow 0$ limit; GP interpolates training data). For a single test input $\mathbf{x}_*$:

$$\bar{f}_* = \mathbf{k}_*^\top K^{-1} \mathbf{f} = \sum_{i=1}^N \alpha_i k(\mathbf{x}_i, \mathbf{x}_*) \quad (3)$$

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top K^{-1} \mathbf{k}_* \quad (4)$$

where $\boldsymbol{\alpha} = K^{-1} \mathbf{f}$, $\mathbf{k}_* = (k(\mathbf{x}_1, \mathbf{x}_*), \dots, k(\mathbf{x}_N, \mathbf{x}_*))^\top$. Noisy form ($K \rightarrow K + \sigma_y^2 I$) on next slide.

Interpretation

Posterior mean is a **weighted sum of kernel basis functions** centered at training points:

$$\bar{f}_* = \sum_{i=1}^N \alpha_i k(\mathbf{x}_i, \mathbf{x}_*)$$

Each observation $\mathbf{x}_i$ “pulls” the prediction via its kernel similarity to $\mathbf{x}_*$.

GP Regression with Noisy Observations

Observation model:

$$y = f(\mathbf{x}) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_y^2)$$

The covariance of *observations* becomes:

$$\text{cov}[y_p, y_q] = k(\mathbf{x}_p, \mathbf{x}_q) + \sigma_y^2 \delta_{pq}$$

In matrix form: replace K with

$$K_y = K + \sigma_y^2 I$$

Effect of noise

- ▶ Predictions no longer interpolate the data exactly.
- ▶ Even at training points, there is **residual uncertainty**.
- ▶ The noise term $\sigma_y^2 I$ also acts as **numerical regularization** (“nugget” / “jitter”).

Noisy GPR: Posterior Equations

Posterior mean and variance with noise:

$$\bar{f}_* = \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{y} \quad (5)$$

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{k}_* \quad (6)$$

Noiseless ($\sigma_y^2 = 0$):

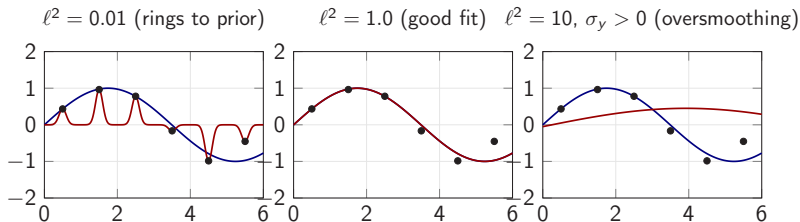
- ▶ Interpolates training points exactly
- ▶ Variance = 0 at training inputs
- ▶ Requires K to be positive definite

Noisy ($\sigma_y^2 > 0$):

- ▶ Smooths through training points
- ▶ Variance > 0 everywhere
- ▶ Better numerical conditioning

In practice: Even for noiseless surrogates, adding a small jitter $\sigma_y^2 \sim 10^{-6}$ improves numerical stability.

The Effect of Kernel Parameters



- ▶ **Blue:** true function $f(x) = \sin(0.9x)$. **Red:** GP posterior mean.
 - ▶ **Too small ℓ (noiseless):** interpolates exactly but the posterior rings back to the prior mean between points $\Rightarrow$ poor predictions away from data.
 - ▶ **Too large ℓ + noise:** GP smooths through the data, misses high-frequency structure of f .
 - ▶ **Bias-variance trade-off in kernel form:** short $\ell \Rightarrow$ low bias but high variance and ringing; long ℓ plus large $\sigma_y^2 \Rightarrow$ high bias and oversmoothing.
- $\Rightarrow$ We need a principled way to learn ℓ , σ_f^2 , and σ_y^2 .

Learning Kernel Hyperparameters

Problem: How to choose ℓ , σ_f , σ_y ? $\Rightarrow$ Maximize the **marginal likelihood**.

Marginal likelihood (model evidence, zero-mean prior)

$$\log p(\mathbf{y} | \mathbf{X}) = \underbrace{-\frac{1}{2} \mathbf{y}^\top K_y^{-1} \mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2} \log |K_y|}_{\text{complexity penalty}} \underbrace{-\frac{N}{2} \log(2\pi)}_{\text{constant}}$$

Intuition:

- ▶ **Data fit** $\uparrow$: reward explaining observations
- ▶ **Complexity** $\downarrow$: penalize overly flexible models
- ▶ Balance = **automatic Occam's razor**

Optimization:

- ▶ Gradient ascent on $\log p(\mathbf{y} | \mathbf{X})$
- ▶ Kernel HPs $\theta_{\text{GP}} = \{\ell, \sigma_f, \sigma_y\}$ (separate from structural θ and NN θ_{NN})
- ▶ Closed-form gradients
- ▶ scikit-learn, GPyTorch

Leave-One-Out Predictive Error

Idea. Out-of-sample predictive accuracy without holding out data.

For a GP on $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with noise σ_y^2 :

$$\mu_{-i}(\mathbf{x}_i) - y_i = -\frac{[K_y^{-1}\mathbf{y}]_i}{[K_y^{-1}]_{ii}}, \quad \sigma_{-i}^2(\mathbf{x}_i) = \frac{1}{[K_y^{-1}]_{ii}}.$$

Why free. Once $\text{diag}(K_y^{-1})$ and $K_y^{-1}\mathbf{y}$ are available from the same Cholesky factor used for posterior inference, all n LOO residuals cost only $\mathcal{O}(n)$ extra. No re-training, no held-out split lost.

Use cases. Surrogate-health diagnostic across active-learning rounds in SMM.

Closed form: Rasmussen & Williams (2006), §5.4.2.

GPR: Numpy Implementation Sketch

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2,1).reshape(-1,1) \
        + np.sum(X2**2,1) - 2*X1@X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8*np.eye(N)
L, low = cho_factor(K, lower=True)    # Cholesky
alpha = cho_solve((L, low), y_tr)    # weights

Ks = kernel(X_tr, X_te)
mu   = Ks.T @ alpha                  # mean
v    = cho_solve((L, low), Ks)
var  = kernel(X_te, X_te) - Ks.T @ v # variance
```

See **Algorithm 2.1** in Rasmussen & Williams (2006), and notebook 02_GP_and_BAL.ipynb.

GP in Practice: scikit-learn & GPyTorch

scikit-learn:

- ▶ GaussianProcessRegressor
- ▶ Automatic hyperparameter tuning via marginal likelihood
- ▶ Great for prototyping ($N \leq 10^4$)
- ▶ CPU only

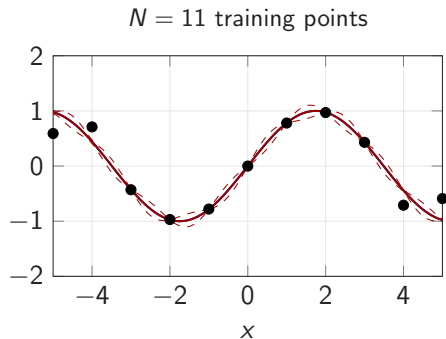
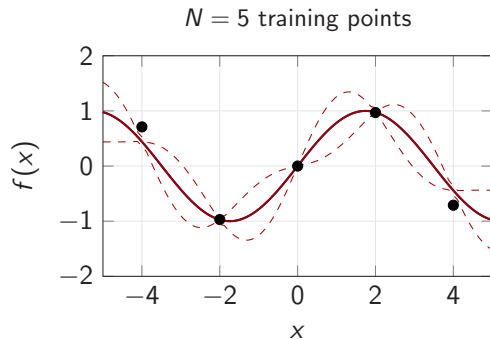
GPyTorch:

- ▶ GPU-accelerated GP inference
- ▶ Scalable to $N > 10^5$ (inducing points, KISS-GP)
- ▶ Integrates with PyTorch ecosystem
- ▶ Production-ready

Hands-on

Notebook: 02_GP_and_BAL.ipynb — GP regression from scratch, scikit-learn, and Bayesian Active Learning.

Prediction & Confidence Intervals



Blue: true function. **Red:** posterior mean. **Dashed:** approximate 95% credible band. The band collapses at training inputs (noiseless GP) and widens between them; with more data the band stays uniformly tight.

Algorithmic Complexity

Cost of GP inference

- ▶ **Training:** Cholesky decomposition of $K_y \in \mathbb{R}^{N \times N}$: $O(N^3)$.
- ▶ **Prediction (per test point, after one-off precomputation of $\alpha = K_y^{-1}y$):** $O(N)$ for mean, $O(N^2)$ for variance.
- ▶ **Hyperparameter optimization:** $O(N^3)$ per gradient step.

Scaling strategies for large N

- ▶ **Sparse GPs:** Inducing point methods — approximate K with rank- M matrix ($M \ll N$), cost $O(NM^2)$.
- ▶ **Random features:** Approximate kernel with random Fourier features.
- ▶ **GPyTorch:** KISS-GP, structured kernel interpolation, GPU acceleration.

For the surrogate problems in this course ($N \leq 10^4$, $d \leq 10$), standard GPs are sufficient.

GPs vs DNNs: When to Use Which

Criterion	Gaussian Processes	Deep Neural Networks
Uncertainty quantification	Built-in (posterior variance)	Requires ensembles / MC dropout
Hyperparameter tuning	Marginal likelihood	Grid search / Bayesian opt.
Scaling with N	$O(N^3)$, limit $\sim 10^4$	$O(N)$ per epoch
Scaling with d	Curse of dimensionality	Handles $d \gg 100$
Small data ($N < 100$)	Excellent	Typically underfits
Training	Closed-form	SGD, many epochs
Interpretability	Kernel encodes prior	Black box

Rule of thumb

- ▶ $d \leq 10$, **need UQ**: GPs (+ BAL).
- ▶ $d > 10$, **large data**: DNNs.

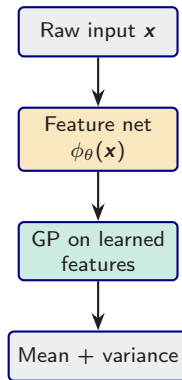
Deep Kernel Learning: Neural Features + GP Uncertainty

Idea: learn a feature map $\phi_{\theta}(\mathbf{x})$ with a neural network, then use

$$k_{\text{DKL}}(\mathbf{x}, \mathbf{x}') = k_{\text{base}}(\phi_{\theta}(\mathbf{x}), \phi_{\theta}(\mathbf{x}'))$$

so the GP operates in the learned feature space.

- ▶ **Front end:** the DNN learns a geometry where the target is smoother.
- ▶ **Back end:** the GP still provides posterior mean and variance.
- ▶ **Training:** optimize the feature map and kernel hyperparameters via the GP marginal likelihood.



When Do Deep Kernels Help?

Standard GP assumption: Euclidean distance in the *raw input space* is informative.

Failure modes:

- ▶ Hidden regimes / thresholds / nonstationarity
- ▶ Inputs are badly warped coordinates of a simple latent state
- ▶ Moderate-to-high-dimensional raw features with strong correlations
- ▶ You still need **uncertainty quantification** or BAL

Rule of thumb

- ▶ If the input is already low-dimensional and smooth: **plain GP** is usually enough.
- ▶ If the latent structure is smooth but the raw geometry is poor: **DKL** is often the right bridge.
- ▶ If d is huge and UQ is not central: **pure DNN surrogate**.

In short: **DKL answers the question “Can we combine DNN feature learning with GP uncertainty?”**

Note on the companion notebook (NB 08). The DKL implementation in NB 08 trains the feature extractor against a hand-crafted teacher feature map for didactic clarity, not jointly with the GP marginal likelihood; see GPyTorch's `DeepKernel` for the joint form.

Summary: Gaussian Process Regression

1. **GPs** = nonparametric Bayesian regression with **built-in UQ**.
2. **Kernel** encodes prior beliefs (smoothness, scale, periodicity).
3. **Posterior**: closed-form mean and variance, conditioned on data.
4. **Hyperparameters**: learned by maximizing the marginal likelihood (automatic Occam's razor).
5. **Limitation**: $O(N^3)$ scaling; best for moderate N and d .
6. **DKL**: neural feature extractor + GP head = useful when raw inputs are warped or moderately high-dimensional, but you still want UQ.

Next: Use the GP's uncertainty to **intelligently select training points** — Bayesian Active Learning.

Motivation: Where to Sample?

Uniform sampling:

- ▶ Places points evenly across the domain
- ▶ Wastes evaluations in smooth regions
- ▶ Cannot adapt to function complexity

Adaptive / active sampling:

- ▶ Concentrates points where the function is **hard to approximate**
- ▶ Uses the model's **uncertainty** to guide selection
- ▶ Fewer evaluations for the same accuracy

Key question

Given a budget of N model evaluations, which input points $\{x_1, \dots, x_N\}$ minimize the overall approximation error?

Bayesian Active Learning: The Idea

Use the GP posterior variance to guide sampling.

A general score function

$$U(\mathbf{x}) = w_{\text{obj}} \cdot \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$$

- ▶ $\sigma^2(\mathbf{x})$: posterior variance — **exploration**; sampling where large reduces global uncertainty.
- ▶ $\mu(\mathbf{x})$: posterior mean — **objective-following** bias (useful in Bayesian optimisation, not for global surrogate accuracy).
- ▶ $w_{\text{obj}}, w_{\text{var}} \geq 0$: fixed weights.

Default: pure exploration ($w_{\text{obj}} = 0$, maximise σ^2 , i.e. uncertainty sampling) for surrogate building. Add the μ -term when also targeting an optimum.

BAL Algorithm

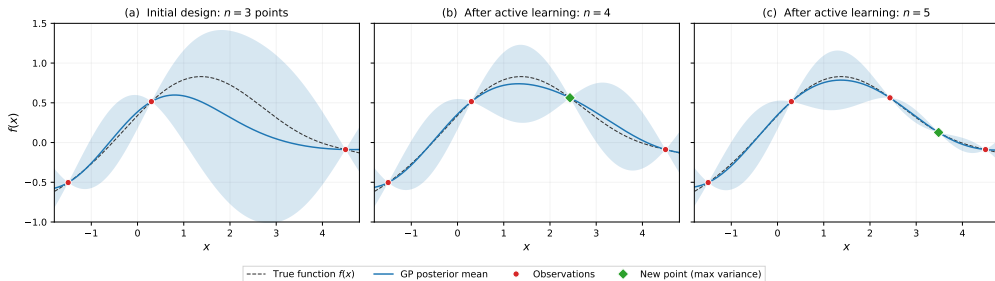
Algorithm 1 Bayesian Active Learning for Surrogates

Require: Initial design $\mathcal{D}_0 = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_0}$, budget N , candidate set $\mathcal{C}$

- 1: Fit GP on $\mathcal{D}_0$: compute posterior mean $\mu(\cdot)$ and variance $\sigma^2(\cdot)$
 - 2: **for** $t = 1, \dots, N - n_0$ **do**
 - 3: Compute score $U(\mathbf{x}) = w_{\text{obj}} \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$
 - 4: Select $\mathbf{x}_{\text{new}} = \arg \max_{\mathbf{x} \in \mathcal{C}} U(\mathbf{x})$
 - 5: Query the expensive model: $y_{\text{new}} = f(\mathbf{x}_{\text{new}})$
 - 6: Update: $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\mathbf{x}_{\text{new}}, y_{\text{new}})\}$
 - 7: Refit GP on $\mathcal{D}_t$
 - 8: **end for**
 - 9: **return** Trained GP surrogate
-

BAL in Action: Progressive Refinement

BAL selects each new point where the **posterior variance is largest** (green diamond), causing the credible band to shrink exactly where it was widest.



(a) 3 initial observations (red dots); GP posterior (blue) deviates in data-sparse regions with wide 95% CI.

(b) Acquisition function picks max-variance point (green diamond); posterior tightens locally.

(c) Second iteration fills the remaining gap — 5 points suffice to track the true function.

Key contrast with uniform: BAL concentrates points where the function is hardest to approximate.

(Pre-generated illustration; notebook 02_GP_and_BAL.ipynb reproduces the BAL loop itself.)

BAL in Economics

Grid-free approximation

BAL + GP = grid-free adaptive sparse grids:

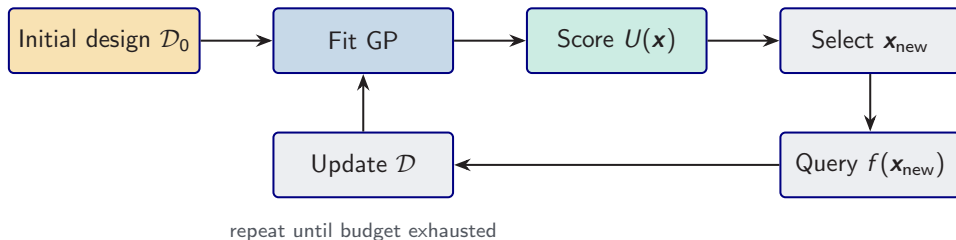
- ▶ No structured grid needed
- ▶ Refinement via posterior variance
- ▶ Scales to $d \leq 10$

Applications

- ▶ **Optimal carbon tax**
- ▶ **Dynamic incentives** (kinks)
- ▶ **Asset pricing** (moderate d)

References: Renner & Scheidegger (2018); Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*).

Combining BAL and GP for Surrogates



Why this combination works

- ▶ **GP provides UQ:** posterior variance tells us where the approximation is uncertain.
- ▶ **BAL selects training points:** maximize information gain per model evaluation.
- ▶ **Surrogate = trained GP:** the same GP used for selection serves as the surrogate.

Ideal for $d \leq 10$ when each model evaluation costs minutes to hours.

Summary: Bayesian Active Learning

1. **BAL** = principled data selection using GP uncertainty.
2. **Score function** balances exploration (high σ^2) and exploitation (high μ).
3. **Faster convergence** than uniform sampling, especially for functions with kinks or local features.
4. **Application:** surrogate construction for moderate-dimensional problems.

Hands-on

Notebook: 02_GP_and_BAL.ipynb — GP from scratch, scikit-learn GP, and BAL comparison.

Recap: The Computational Toolbox

PINNs

- ▶ Continuous-time PDEs
- ▶ Physics loss
- ▶ Mesh-free
- ▶ HJB, Black–Scholes

Deep Surrogates

- ▶ Discrete-time
- ▶ Pseudo-states
- ▶ Fast evaluation
- ▶ Estimation, UQ

GPs + BAL

- ▶ Nonparametric
- ▶ Built-in UQ
- ▶ Active learning
- ▶ Moderate dim

All three approaches are **complementary**: choose based on the problem structure, dimensionality, whether UQ is needed, and available hardware.

PINNs are not covered in this school; see the lecture script for a primer.

The Toolbox: When to Use What

Method	Best for	Key strength
DEQN	Discrete-time equilibrium models	Scales to high d , Euler eq. loss
PINN	Continuous-time PDEs/HJB	Mesh-free, physics-constrained
Deep Surrogate	Fast evaluation, estimation, UQ	Pseudo-states, autograd
GP + BAL	Moderate d , need UQ	Built-in uncertainty, active learning
Deep Kernel	Warped inputs + need UQ	Learned features + GP uncertainty

Decision heuristic

1. Is it a continuous-time PDE? $\Rightarrow$ **PINN**.
2. Do you need to re-evaluate for many parameter values? $\Rightarrow$ **Deep Surrogate**.
3. Is $d \leq 10$ and you need UQ / active learning? $\Rightarrow$ **GP + BAL**.
4. Is the latent structure smooth, but the raw input geometry is poor? $\Rightarrow$ **Deep Kernel**.
5. Is it a discrete-time equilibrium model? $\Rightarrow$ **DEQN**.

References & Resources

Key papers:

- ▶ Chen, Didisheim & Scheidegger (2026). *Deep Surrogates for Finance: With an Application to Option Pricing*. J. Financial Econ.
- ▶ Scheidegger & Bilonis (2019). *ML for High-Dim Dynamic Stochastic Economies*. J. Comput. Sci.
- ▶ Kübler, Scheidegger & Surbek (2026). *Using Machine Learning to Compute Constrained Optimal Carbon Tax Rules*. Forthcoming in *JPE Macroeconomics*.
- ▶ Rasmussen & Williams (2006). *GP for ML*. MIT Press.
- ▶ Wilson et al. (2016). *Deep Kernel Learning*. AISTATS.

Notebooks (in `lectures/lecture_2/code/`): 01_Surrogate_Primer, 02_GP_and_BAL, 03_Deep_Kernel_Learning.

GitHub: https://github.com/sischei/cemracs2026_DEQN

Questions?